JAVA反射机制

配置文件代码：在实际的应用开发中，一般将具体工厂类的实例化过程进行改进， 不直接使用new关键字来创建对象，而是将具体类的类名写入配置文件中，再通过**Java的反射机制**，读取XML格式的配置文件，根据 存储在XML文件中的类名字符串生成对象。

```xml
<?xml version="1.0"?>
<config>
    <className>CashPayFactory</className>
</config>
```

Java反射(Java Reflection)： 是指在程序运行时获取已知名称的类或已有对象的相关信息的一种机制，包括类的方法、属性、超类等信息，还包括实例的创建和实例类型的判断等。可**通过Class类的forName()方法返回与带有给定字符串名的类或接口相关联的Class对象**，再通过newInstance()方法创建此对象所表示的类的一个新实例，即通过一个类名字符串得到类的实例。

```java
//创建一个字符串类型的对象
Class c = Class.forName("String");
Object obj = c.newInstance();
return obj;
```

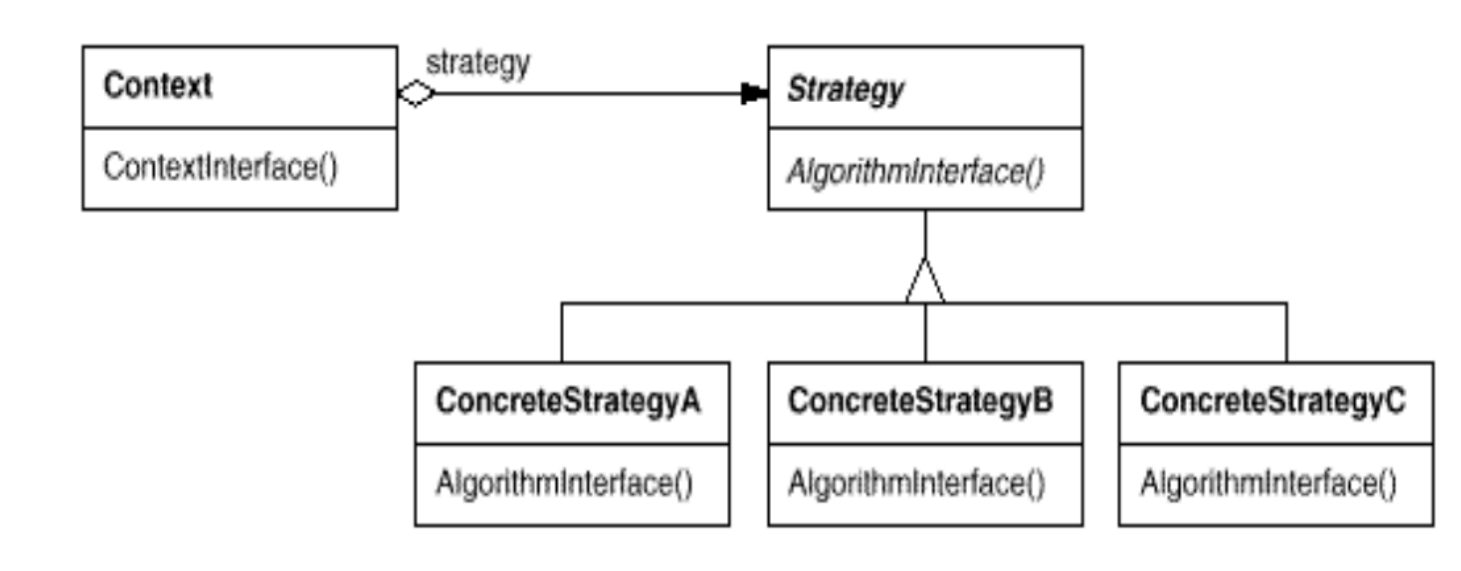
工具类XMLUtil代码片段

```java
//创建DOM文档对象
DocumentBuilderFactory dFactory = DocumentBuilderFactory.newInstance();
DocumentBuilder builder = dFactory.newDocumentBuilder();
Document doc;
doc = builder.parse(new File("config.xml"));
//获取包含类名的文本节点
NodeList nl = doc.getElementsByTagName("className");
Node classNode=nl.item(0).getFirstChild();
String cName=classNode.getNodeValue();
//通过类名生成实例对象并将其返回
Class c=Class.forName(cName);
Object obj=c.newInstance();
return obj;
```

各个设计模式演变的原因

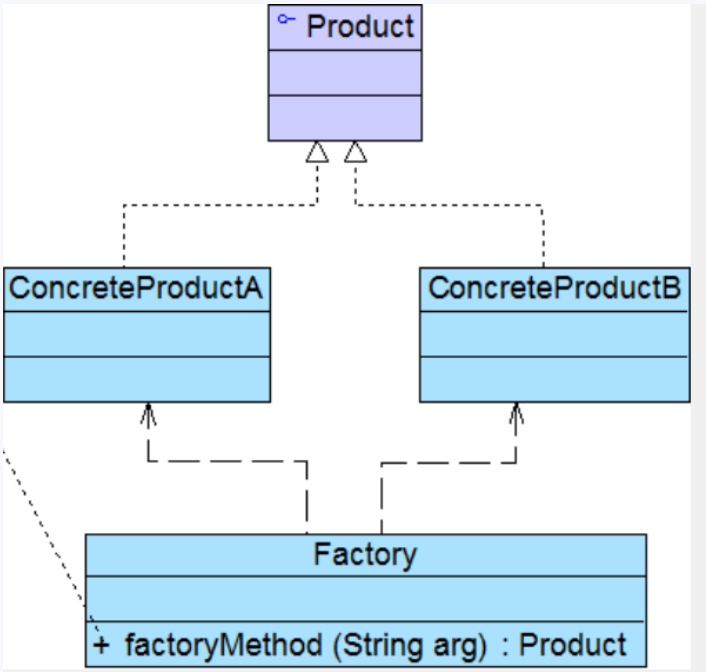
1. 同一件事情有很多种做法，为每一个做法创建一个类，解决办法的时候分别调用这些类，很麻烦，因此产生了策略模式
2. 为了不让客户端调用的代码里面出现底层实现也就是new 具体的类，所以才产生了简单工厂模式
3. 简单工厂模式无法实现开闭原则，因为一个工厂会创建很多不同的产品，所以如果增加一个产品那么就要修改当前的工厂类，所以产生了工厂模式
4. 如果一个产品会有不同实现方式，比如一个产品会有两个产品生产方式，正常的工厂模式就无法解决了

设计模式一--策略模式



设计模式二--简单工厂模式

```
if(arg.equalsIgnoreCase("A"))
{
    return new ConcreteProductA();
}
else if(arg.equalsIgnoreCase("B"))
{
    return new ConcreteProductB();
}
else
{
    ......
}
```

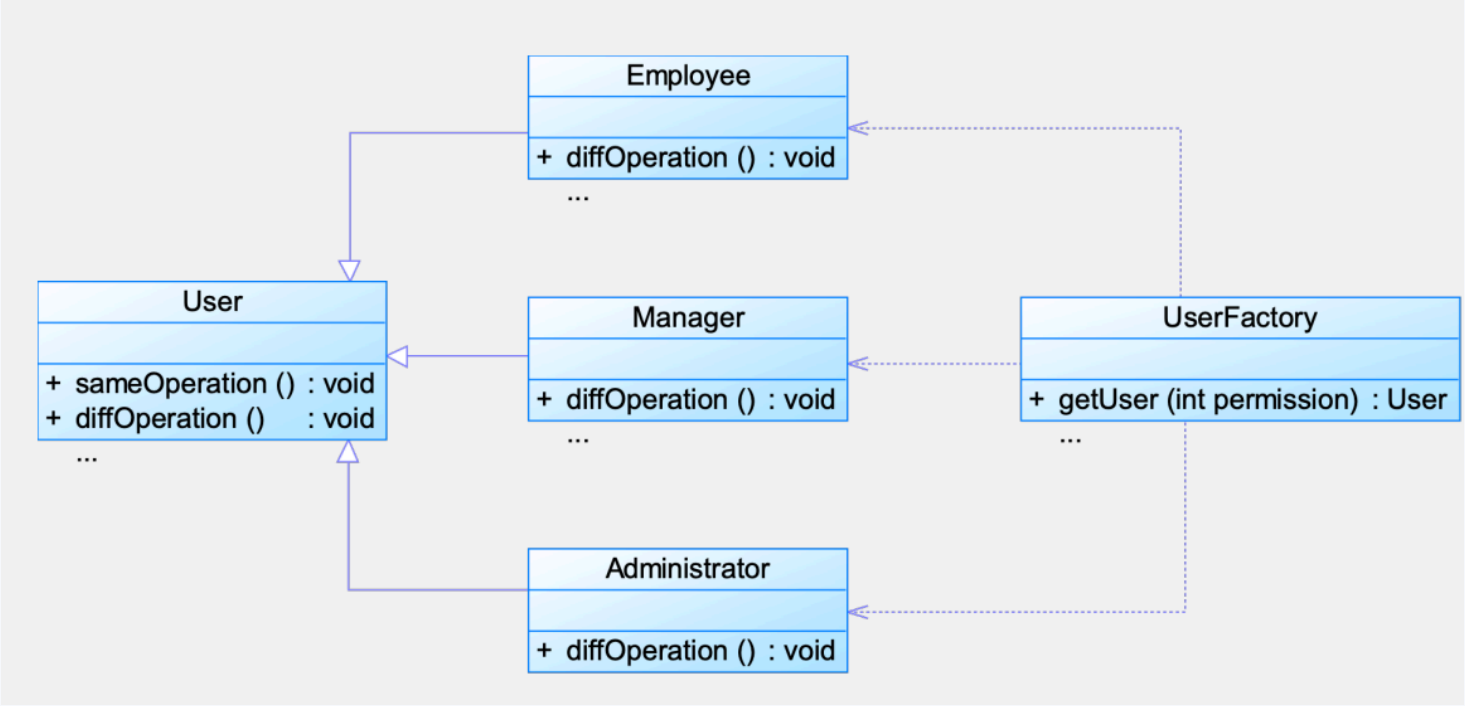


方法是**静态**的　**记得加static**

简单工厂模式包含如下角色：

- **Factory：工厂角色**
- **Product：抽象产品角色**

- **ConcreteProduct：具体产品角色**

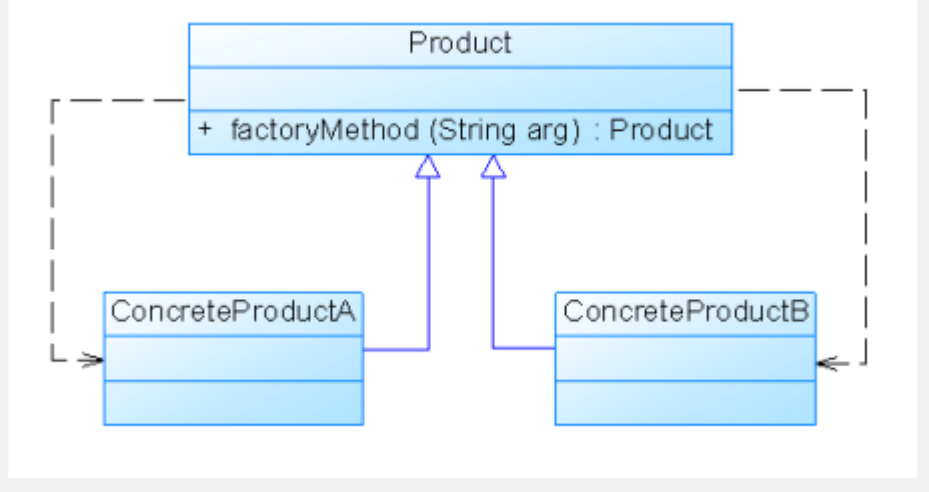



```java
public abstract class AbstractPay{
    public abstract void pay();
}
public class CashPay extends AbstractPay{
    public void pay(){
    //现金支付处理代码
    }
}
public class PayMethodFactory
{
    public static AbstractPay getPayMethod(String type){
        if(type.equalsIgnoreCase("cash")){
            return new CashPay(); //根据参数创建具体产品
        }else if(type.equalsIgnoreCase("creditcard")){
            return new CreditcardPay(); //根据参数创建具体产品
        }
        ……
    }
}
```

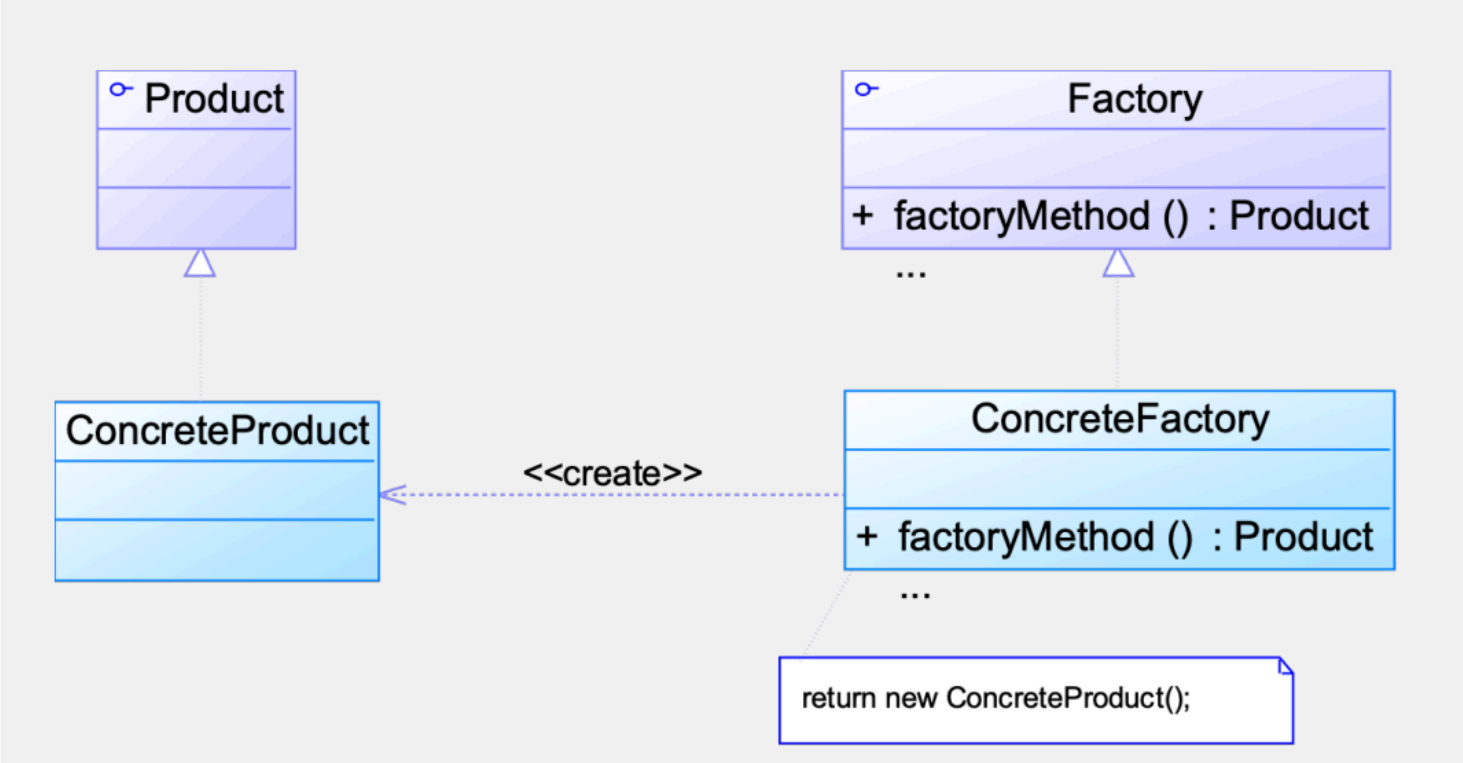
模式扩展

• 简单工厂模式的简化：

• 在有些情况下工厂类可以由抽象产品角色扮演，一 个抽象产品类同时也是子类的工厂，也就是说把静 态工厂方法写到抽象产品类中



设计模式三--工厂模式

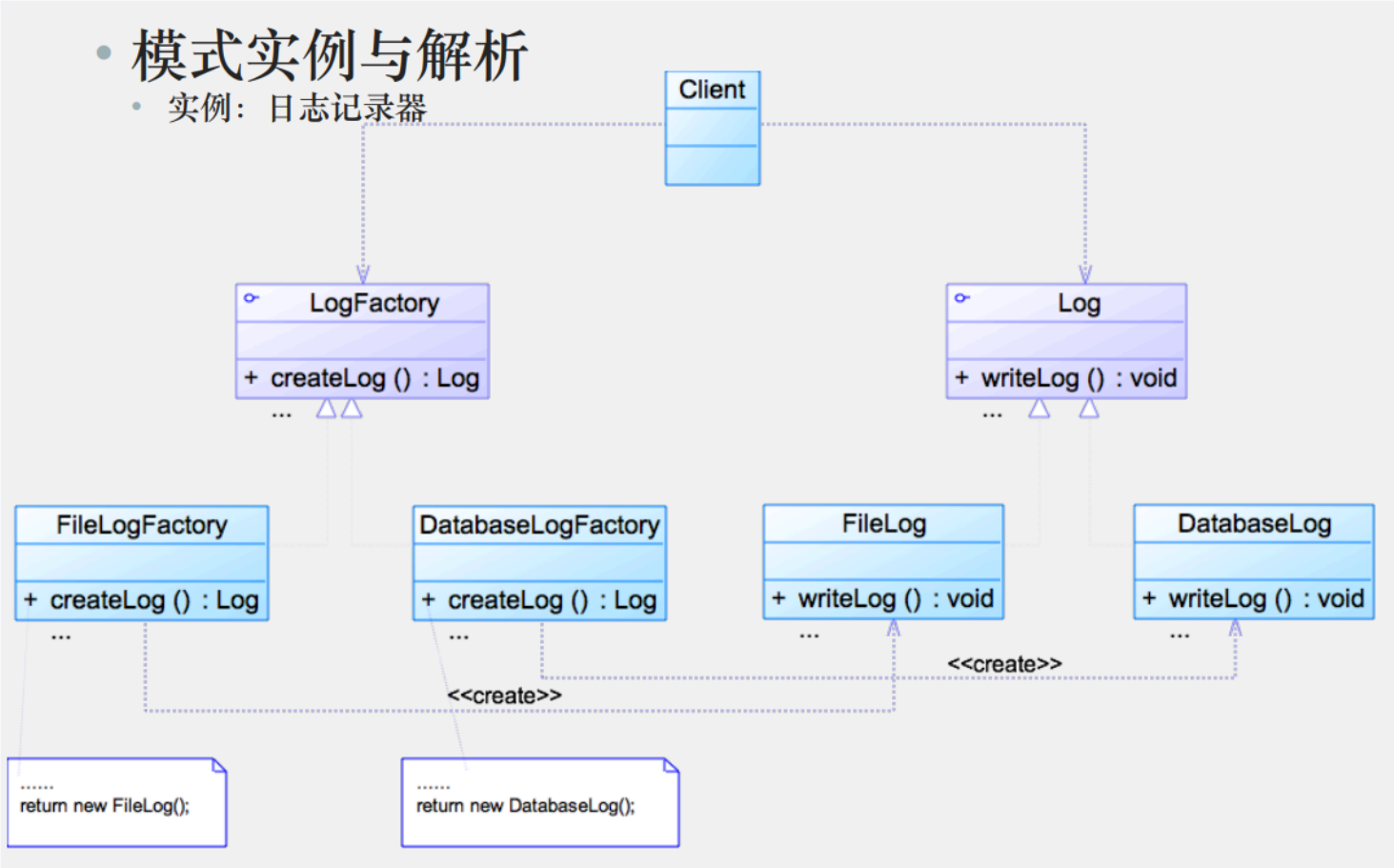


工厂模式包含如下角色：

- **Factory：抽象工厂角色（一个）**
- **ConcreteFactory：具体工厂角色（多个）**

- **Product：抽象产品角色（一个）**
- **ConcreteProduct：具体产品角色（多个）**

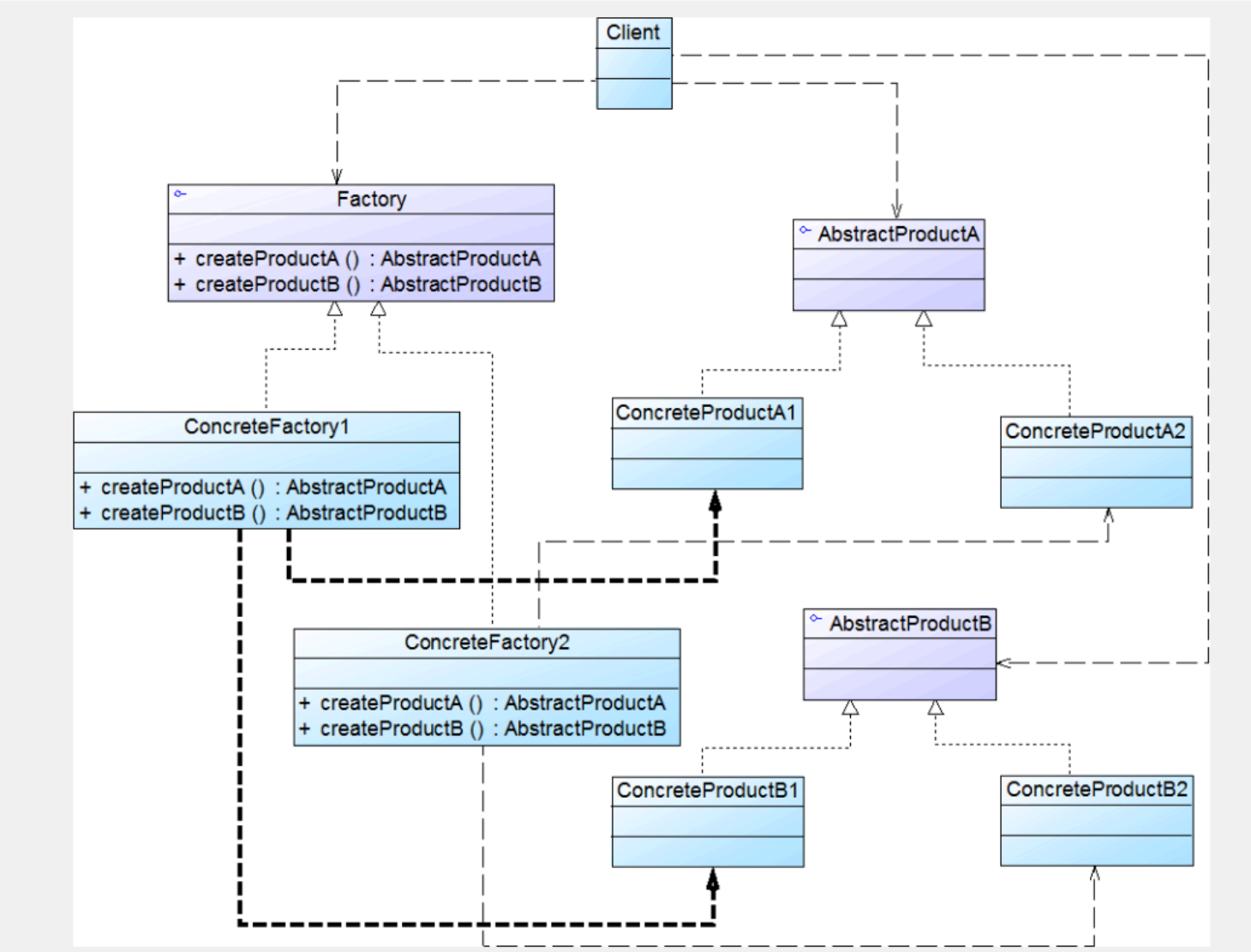
```java
public abstract class PayMethodFactory{
    public abstract AbstractPay getPayMethod();
}
public class CashPayFactory extends PayMethodFactory{
    public AbstractPay getPayMethod(){
        return new CashPay();
    }
}

//客户端
PayMethodFactory factory;
AbstractPay payMethod;
factory=new CashPayFactory();
payMethod =factory.getPayMethod();
payMethod.pay();

//客户端也出现了CashPayFactory现在不想出现
PayMethodFactory factory;
AbstractPay payMethod;
factory=(PayMethodFactory)XMLUtil.getBean(); //getBean()的返回类型为Object，此处需要进行强制类型转换
payMethod =factory.getPayMethod();
payMethod.pay();
```



设计模式四--抽象工厂模式

Abstract Factory Pattern: Provide an interface for creating **families of related or dependent objects** without specifying their concrete classes.

我们需要一个工厂可以提供多个产品对象，而不是单一的产品对象。

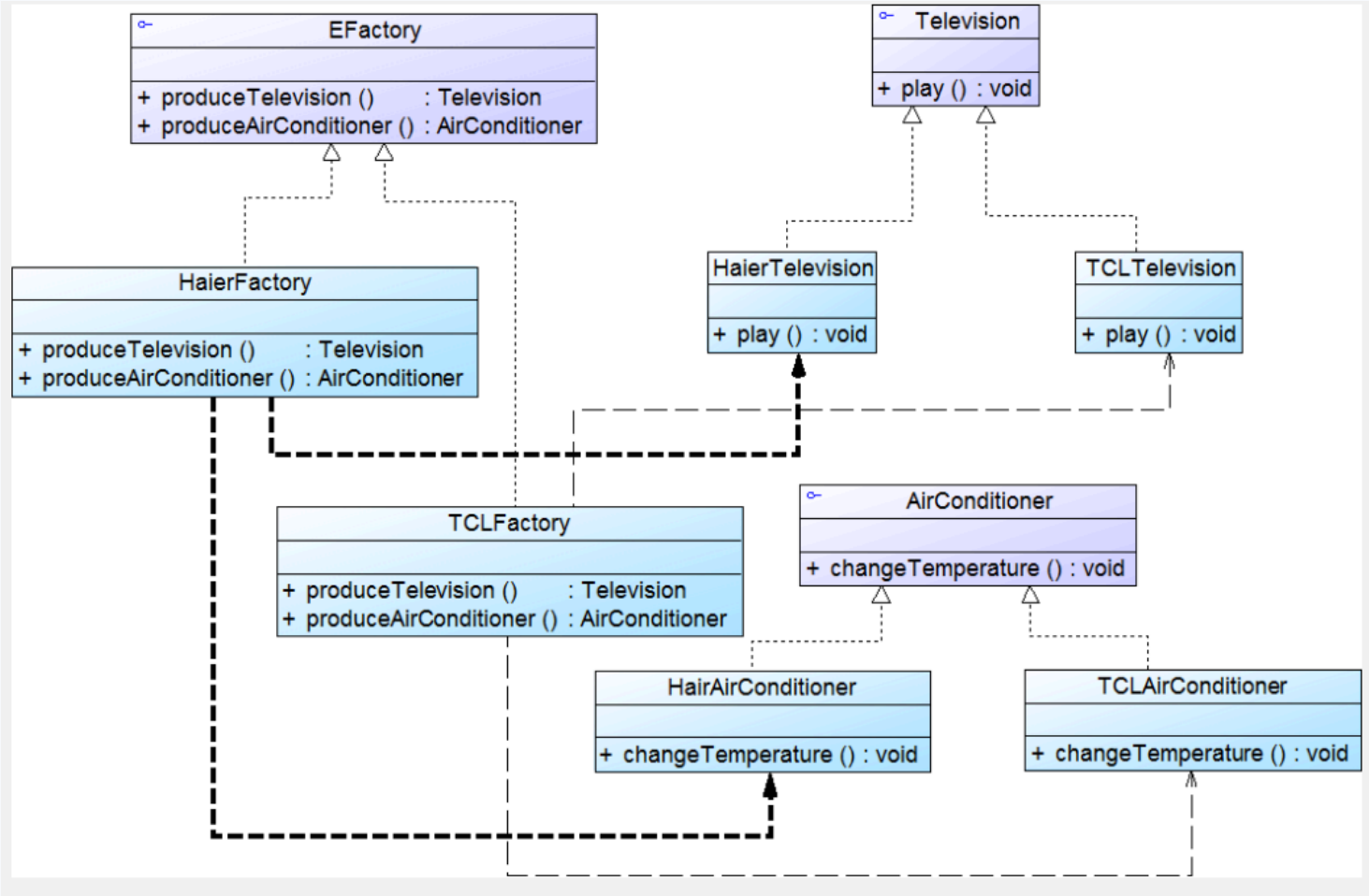


工厂模式包含如下角色：

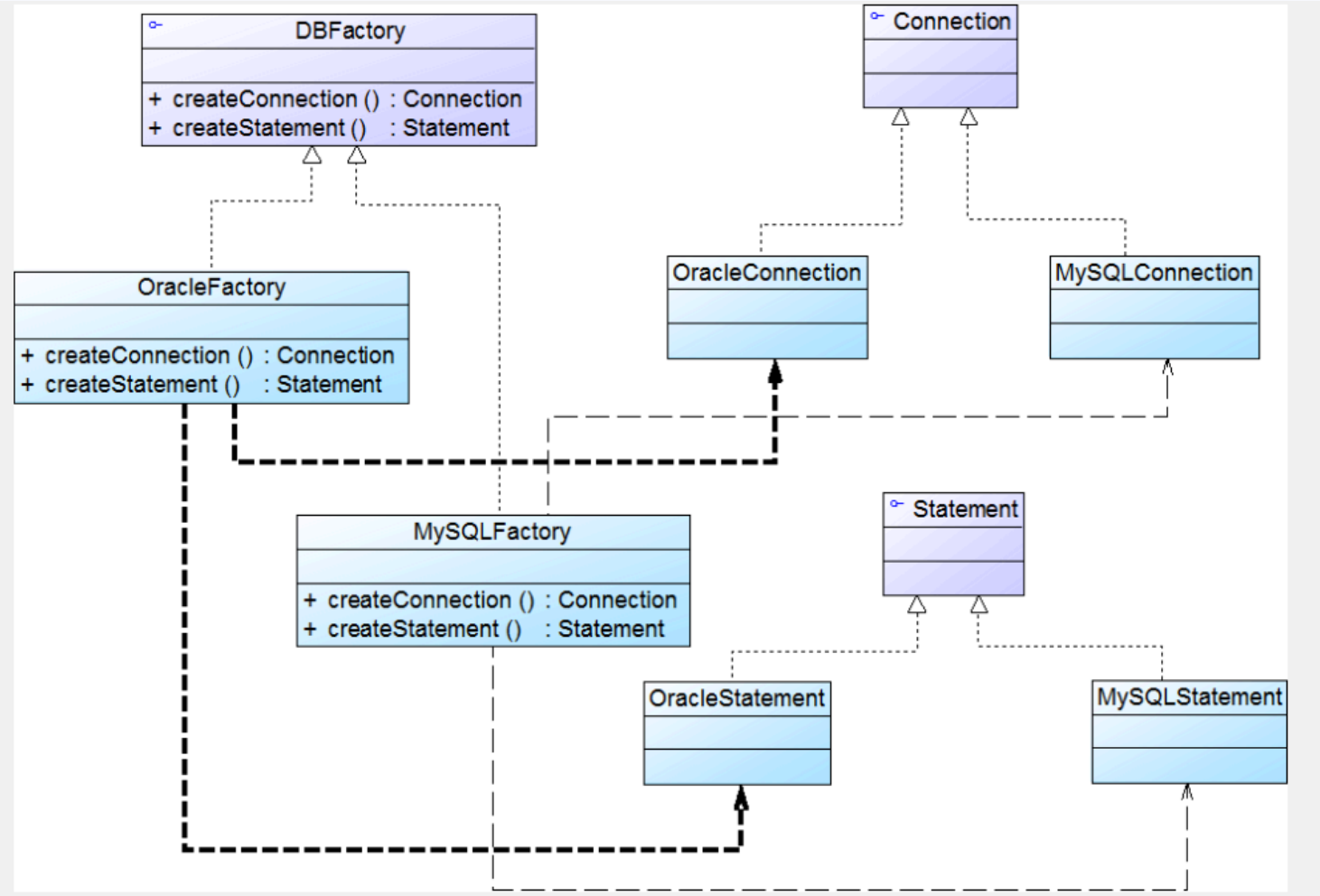
- **AbstractFactory：抽象工厂角色（一个）**
- **ConcreteFactory：具体工厂角色（多个）**
- **AbstractProduct：抽象产品角色（多个）**
- **ConcreteProduct：具体产品角色（多个）**

一个抽象产品会对多个具体产品这个就是抽象工厂和工厂的区别

一个电器工厂可以产生多种类型的电器，如海尔工厂 可以生产海尔电视机、海尔空调等，TCL工厂可以生产 TCL电视机、TCL空调等，相同品牌的电器构成一个产品 族，而相同类型的电器构成了一个产品等级结构，现使用 抽象工厂模式模拟该场景。



某系统为了改进数据库操作的性能，自定义数据库连 接对象Connection和语句对象Statement，可针对不同类 型的数据库提供不同的连接对象和语句对象，如提供 Oracle或SQL Server专用连接类和语句类，而且用户可以 通过配置文件等方式根据实际需要动态更换系统数据库。 使用抽象工厂模式设计该系统。




```java
public abstract class AbstractFactory{
    public abstract AbstractProductA createProductA();
    public abstract AbstractProductB createProductB();
}
public class ConcreteFactory1 extends AbstractFactory{
    public AbstractProductA createProductA(){
        return new ConcreteProductA1();
}
public AbstractProductB createProductB(){
    return new ConcreteProductB1();
    }
}
```

抽象工厂总结：通过factory选择方式，通过factory调用的方法选择产品

作业

- 设计一个披萨店点餐系统，能够根据顾客选择的披萨类型创建相应的披萨对象。
- 要求：
- 1. 定义一个抽象类 Pizza，包含披萨的基本属性和方法，例如：prepare() 准备披萨；bake()烘烤披萨；cut()切割披萨；box() 打包披萨等。
- 2. 定义三种披萨子类NewYorkPizza（纽约风味披萨），ChicagoPizza （芝加哥风味披萨），CaliforniaPizza （加州风味披萨）。
- 3. 使用工厂方法模式实现这三种披萨的实例化，并画出相应的UML类图。

```
public abstract class Pizza{
    public abstract void prepare();
    public abstract void bake();
    public abstract void cut();
    public abstract void box();
}
 class NewYorkPizza extends Pizza{
    public void prepare(){}
    public void bake(){}
    public void cut(){}
    public void box(){}
}
 class ChicagoPizza extends Pizza{
    public void prepare(){}
    public void bake(){}
    public void cut(){}
    public void box(){}
}
 class CaliforniaPizza extends Pizza{
```

```
20      public void prepare(){}
21      public void bake(){}
22      public void cut(){}
23      public void box(){}
24  }
25
26
27
28
29
30  public abstract class PizzaChooseFactory{
31      public abstract Pizza getPizza();
32  }
33   class NewYorkPizzaChooseFactory extends PizzaChooseFactory{
34      public Pizza getPizza(){
35          return new NewYorkPizza();
36      }
37  }
38   class ChicagoPizzaChooseFactory extends PizzaChooseFactory{
39      public Pizza getPizza(){
40          return new ChicagoPizza();
41      }
42  }
43   class CaliforniaPizzaChooseFactory extends
    PizzaChooseFactory{
44      public Pizza getPizza(){
45          return new CaliforniaPizza();
46      }
47  }
48
49
50
51
52  public static void main(String[] args){
53      PizzaChooseFactory factory = new
    ChicagoPizzaChooseFactory();
54      Pizza p = factory.getPizza();
55      p.prepare();
56      p.bake();
```

```
57      p.cut();
58      p.box();
59
60  }
```